

**Diseño e Implementación de un Agente en Espacio de
Usuario**
**para la Gestión Dinámica de Frecuencia (DVFS) en
Sistemas**
**Heterogéneos mediante Modelos Ligeros de Machine
Learning**

Yeison Adrián Cáceres Torres

Ricardo Andrés Pérez Porras

Trabajo de Grado para Optar el Título de Ingeniero de Sistemas

Director

Gilberto Javier Díaz Toro

Doctor en Ingeniería

Universidad Industrial de Santander

Facultad de Ingenierías Físico-Mecánicas

Escuela de Ingeniería de Sistemas e Informática

Bucaramanga

2026

Agradecimientos

Resumen

Título: Diseño e Implementación de un Agente en Espacio de Usuario para la Gestión Dinámica de Frecuencia (DVFS) en Sistemas Heterogéneos mediante Modelos Ligeros de Machine Learning.

Autores: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Palabras Clave: DVFS; eficiencia energética; computación de alto rendimiento; modelo Roofline; telemetría microarquitectónica; aprendizaje automático supervisado.

Descripción:

El consumo energético constituye hoy una restricción estructural para el escalamiento de la computación de alto rendimiento (HPC), particularmente en nodos heterogéneos que integran CPU y aceleradores gráficos. El Escalado Dinámico de Voltaje y Frecuencia (DVFS) es el principal mecanismo de control disponible a nivel de hardware, pero su aprovechamiento efectivo depende de decidir, durante la ejecución, cuál punto operativo conviene según el comportamiento de la carga. Los gobernadores nativos del sistema operativo deciden a partir de la utilización agregada del procesador, una señal que no distingue si el rendimiento está limitado por la capacidad aritmética del núcleo o por la transferencia de datos desde memoria; en el segundo caso, sostener la frecuencia máxima incrementa el consumo sin una mejora proporcional del tiempo de ejecución.

Este trabajo propone el diseño, la implementación y la evaluación de un agente en espacio de usuario que infiere el régimen de ejecución de la aplicación a partir de telemetría microarquitectónica y traduce esa inferencia en decisiones discretas de DVFS sobre CPU y GPU, con el objetivo de optimizar el Producto Energía–Retardo (EDP) sin incurrir en una penalización severa del rendimiento ni en una sobrecarga de inferencia que anule el beneficio

energético.

El desarrollo se organiza en cuatro fases: caracterización de cargas y construcción del conjunto de datos; entrenamiento y selección de un clasificador supervisado ligero; implementación del servicio de control; y validación experimental frente a los gobernadores nativos de Linux. El presente documento reporta el instrumento de medición y el protocolo experimental contruidos durante la primera fase — cuyo aporte metodológico central es un procedimiento reproducible para etiquetar como *compute-bound* o *memory-bound* cada ventana de ejecución de CPU o corrida completa de GPU, mediante una calibración empírica del modelo Roofline sobre la plataforma de medición, con trazabilidad explícita de la calidad de cada observación — junto con un primer entrenamiento y análisis del clasificador de GPU, posible ya dentro de esta fase por disponer de su conjunto de entrenamiento definido.

Abstract

Title: Design and Implementation of a User-Space Agent for Dynamic Frequency Management (DVFS) in Heterogeneous Systems through Lightweight Machine Learning Models.

Authors: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Keywords: DVFS; energy efficiency; high performance computing; Roofline model; micro-architectural telemetry; supervised machine learning.

Description:

Energy consumption has become a structural constraint on the scaling of high performance computing (HPC), particularly on heterogeneous nodes that combine CPUs and graphics accelerators. Dynamic Voltage and Frequency Scaling (DVFS) is the main control mechanism available at the hardware level, but exploiting it effectively requires deciding, at run time, which operating point suits the current workload behaviour. Native operating system governors base that decision on aggregate processor utilisation, a signal that does not distinguish whether performance is limited by the arithmetic capability of the core or by data transfer from memory; in the latter case, sustaining maximum frequency raises power draw without a proportional improvement in execution time.

This work proposes the design, implementation and evaluation of a user-space agent that infers the execution regime of an application from microarchitectural telemetry and translates that inference into discrete DVFS decisions over CPU and GPU, aiming to optimise the Energy-Delay Product (EDP) without incurring a severe performance penalty or an inference overhead that cancels the energy benefit.

The work is organised in four phases: workload characterisation and dataset construction; training and selection of a lightweight supervised classifier; implementation of the control

service; and experimental validation against the native Linux governors. This document reports the measurement instrument and experimental protocol built during the first phase, whose central methodological contribution is a reproducible procedure for labelling execution windows as *compute-bound* or *memory-bound* through an empirical calibration of the Roofline model on the measurement platform, with explicit traceability of the quality of each observation.

Índice general

Agradecimientos	1
Resumen	2
Abstract	4
Introducción	9
Planteamiento y justificación del problema	10
Objetivos	11
1. Marco de referencia	13
1.1. Marco Conceptual	13
1.1.1. Potencia CMOS y fundamento de DVFS	13
1.1.2. Modelo Roofline y regímenes de ejecución	14
1.1.3. Telemetría microarquitectónica con PMU	16
1.1.4. Espacios de ejecución <i>user-space</i> y <i>kernel-space</i>	18
1.1.5. Interfaces de observabilidad	18
1.1.6. DVFS, P-states y mecanismos de actuación	19
1.1.7. Cargas de calibración y modelos de clasificación ligera	20
1.1.8. Hiperparámetros, Optuna y validación	21
1.1.9. Producto Energía-Retardo (EDP)	22
1.2. Estado del Arte	22
Repositorio y recursos complementarios	30

Índice de tablas

Índice de figuras

1.1. Modelo Roofline conceptual	14
1.2. Unidad de Monitorización de Rendimiento (PMU)	16
1.3. Contadores de núcleo y contadores de <i>uncore</i>	17
1.4. Espacios de usuario y de kernel	18
1.5. P-states, C-states y jerarquía DVFS	19

Introducción

En los sistemas de cómputo de alto rendimiento (HPC), el consumo energético se ha consolidado como una restricción crítica para el escalado sostenible, especialmente en arquitecturas heterogéneas CPU–GPU. En este contexto, el Escalado Dinámico de Voltaje y Frecuencia (DVFS) constituye el principal mecanismo de control a nivel de hardware, permitiendo ajustar los estados operativos de los procesadores para optimizar el compromiso entre consumo energético y tiempo de ejecución. Sin embargo, la efectividad de este mecanismo depende de la capacidad del sistema para adaptar dinámicamente la frecuencia al comportamiento cambiante de las aplicaciones, lo cual representa un desafío abierto en entornos HPC modernos.

En la literatura, este problema ha sido abordado principalmente mediante dos enfoques. Por un lado, los gobernadores de energía del sistema operativo, como *ondemand* o *schedutil*, emplean políticas reactivas basadas principalmente en la utilización del procesador, sin considerar la naturaleza microarquitectónica de la carga de trabajo, lo que limita su capacidad de adaptación a cargas de trabajo científicas dinámicas. Por otro lado, diversas propuestas han explorado estrategias heurísticas basadas en métricas microarquitectónicas, utilizando reglas deterministas para ajustar la frecuencia de operación. Aunque estos métodos presentan bajo *overhead*, su dependencia de umbrales estáticos y su limitada capacidad de generalización reducen su efectividad en escenarios donde las fases computacionales varían rápidamente.

A pesar de estos avances, persiste una limitación fundamental: la incapacidad de los enfoques actuales para capturar de forma robusta las transiciones dinámicas entre regímenes *compute-bound* y *memory-bound*, así como para adaptarse a la heterogeneidad CPU–GPU sin intervención manual o ajuste específico por carga de trabajo. Esta brecha evidencia la necesidad de mecanismos de control más flexibles que permitan inferir el régimen computacional de la aplicación en tiempo de ejecución y tomar decisiones de escalado de frecuencia fundamentadas en el perfil de ejecución.

En este contexto, el presente trabajo propone el diseño e implementación de un agente en

espacio de usuario basado en modelos ligeros de aprendizaje automático, capaz de clasificar en tiempo de ejecución las fases de una aplicación como compute-bound o memory-bound a partir de telemetría microarquitectónica, y de ajustar dinámicamente la frecuencia de CPU y GPU para optimizar el Producto Energía–Retardo (EDP). Se plantea demostrar que este enfoque mejora la eficiencia energética frente a los gobernadores nativos de Linux, manteniendo una degradación controlada del rendimiento y una sobrecarga de inferencia mínima y cuantificada mediante métricas experimentales.

Planteamiento y justificación del problema

Existe una ineficiencia energética estructural en la operación de los sistemas de alto rendimiento (HPC), especialmente en infraestructuras que integran aceleradores gráficos (GPU) para ejecutar aplicaciones científicas intensivas. Cuando una aplicación entra en una fase donde el procesamiento depende principalmente de la transferencia de datos desde memoria, mantener la CPU o la GPU a su frecuencia máxima no mejora el rendimiento de manera proporcional. En estas condiciones, los núcleos de cómputo permanecen parcialmente inactivos esperando datos, lo que genera un consumo energético elevado sin un beneficio equivalente en tiempo de ejecución. Esta desalineación entre demanda computacional y frecuencia operativa ha motivado el estudio del escalado de frecuencia como mecanismo para mejorar la eficiencia energética en aplicaciones científicas [29] y en sistemas de gran escala donde la gestión de potencia impacta el comportamiento global del sistema [10].

El problema se agrava por la naturaleza multifásica de las aplicaciones HPC, cuyo perfil de ejecución cambia a lo largo del tiempo de ejecución. En particular, estas aplicaciones alternan entre fases donde el rendimiento está limitado por la capacidad de cómputo del procesador (*compute-bound*) y fases donde está restringido por la transferencia de datos desde memoria (*memory-bound*). Esta distinción es crítica porque, en el régimen *memory-bound*, incrementos de frecuencia tienden a producir mejoras marginales en el tiempo de ejecución, mientras aumentan el consumo energético; en contraste, en fases *compute-bound* la frecuencia sí impacta el desempeño de forma más directa. Por ello, se han propuesto marcos de caracterización para identificar y clasificar estas fases en entornos HPC, con el fin de habilitar decisiones de control más informadas [4, 7].

A partir de este contexto, la gestión eficiente de frecuencia no depende únicamente de la disponibilidad del mecanismo DVFS en el hardware, sino de la capacidad del sistema para decidir, en tiempo de ejecución, cuál configuración resulta más conveniente según el comportamiento de la carga. Aunque se han propuesto mecanismos automatizados para seleccionar

frecuencias óptimas de GPU con el fin de mejorar la eficiencia energética sin comprometer significativamente el desempeño [2], persiste una brecha en la implementación de estrategias que integren de forma coordinada componentes CPU–GPU y que operen durante la ejecución de la aplicación, considerando además el costo computacional asociado al propio proceso de decisión.

Esta brecha resulta especialmente relevante en entornos académicos y científicos, donde el consumo eléctrico incide directamente en los costos operativos, la disponibilidad efectiva de recursos y la sostenibilidad institucional. En este contexto, contar con un mecanismo capaz de observar el comportamiento de la aplicación, inferir su fase de ejecución y ajustar la frecuencia de operación sin intervenir el código fuente constituye una alternativa con valor técnico y práctico.

Desde la perspectiva internacional, la literatura reciente ha mostrado avances en la optimización energética mediante escalado de frecuencia, gestión de potencia y caracterización de cargas HPC [29, 10, 4, 7, 2]. Sin embargo, a nivel nacional y regional, este tipo de soluciones aún no se encuentra ampliamente implementado en infraestructuras académicas de cómputo científico, particularmente bajo esquemas que combinen telemetría de bajo nivel, modelos ligeros de aprendizaje automático y control en espacio de usuario. En consecuencia, el problema no solo es técnicamente relevante, sino también pertinente en términos de aplicabilidad local.

A partir de esta necesidad de optimización energética en sistemas heterogéneos, se formula la siguiente pregunta de investigación que servirá como eje de desarrollo del proyecto:

¿En qué medida el diseño e implementación de un agente en espacio de usuario, guiado por modelos ligeros de aprendizaje automático, permite ajustar la frecuencia de operación (DVFS) en sistemas heterogéneos CPU–GPU para reducir el consumo energético, sin que el costo de la inferencia computacional degrade el rendimiento global de la aplicación, en comparación con los gobernadores nativos de Linux?

Objetivos

Objetivo General

Caracterizar, diseñar, implementar y evaluar un agente en espacio de usuario basado en modelos ligeros de Aprendizaje Automático para el ajuste dinámico de frecuencia (DVFS)

en sistemas heterogéneos CPU–GPU, con el propósito de maximizar la eficiencia energética en aplicaciones científicas sin inducir una penalización severa en su rendimiento global.

Objetivos Específicos

1. Caracterizar el comportamiento computacional y el consumo energético de cargas de trabajo representativas (intensivas en cómputo y en memoria) bajo distintos estados de frecuencia, recolectando telemetría de bajo nivel mediante contadores de rendimiento por hardware e interfaces de potencia estándar (Perf y RAPL para CPU y NVML para GPU).
2. Entrenar y validar modelos clásicos de Aprendizaje Automático (tales como Árboles de Decisión o Bosques Aleatorios) que sean capaces de clasificar, en tiempo de ejecución y con baja latencia de inferencia, las fases de ejecución de las aplicaciones basándose en la telemetría extraída.
3. Desarrollar un servicio de control (*daemon*) en el espacio de usuario que interactúe de forma asíncrona con el sistema operativo, leyendo el estado de los contadores de hardware, ejecutando la inferencia del modelo clasificador y aplicando políticas proactivas de DVFS a través de las interfaces estándar del sistema operativo, en función de la fase de ejecución inferida por el modelo.
4. Evaluar el impacto empírico del sistema propuesto mediante el cálculo del Producto Energía–Retardo (EDP), con el fin de determinar estadísticamente si el ahorro energético compensa la sobrecarga computacional (*overhead*) derivado de la inferencia del modelo en comparación con los gobernadores nativos de Linux.

Capítulo 1

Marco de referencia

1.1. Marco Conceptual

El marco conceptual parte de la relación entre el ajuste de la frecuencia y del voltaje (DVFS), la potencia requerida por el hardware y el tiempo de ejecución, cuyo comportamiento depende del régimen que limita cada carga. Régimen que puede estar condicionado por capacidad de cómputo (compute-bound) o de memoria (memory-bound), dos límites que se pueden distinguir mediante el modelo Roofline y algunas señales del sistema asociadas con este comportamiento durante la ejecución. Señales que funcionan como vector de entrada a un clasificador ligero que intenta inferir el régimen, permitiendo así tomar decisiones de ajuste de frecuencia, teniendo en cuenta el Producto Energía–Retardo (EDP), el cual permite relacionar el consumo energético con el tiempo de ejecución para valorar el efecto de dichas decisiones.

1.1.1. Potencia CMOS y fundamento de DVFS

La potencia dinámica de un circuito CMOS depende del factor de actividad, la capacitancia efectiva, el voltaje de alimentación y la frecuencia de operación. Esta relación es la base física de DVFS y se expresa como:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f \quad (1.1)$$

donde α representa el factor de actividad, C la capacitancia efectiva conmutada, V el voltaje de alimentación y f la frecuencia de operación [13]. El voltaje y la frecuencia están relacionados dentro de los estados de rendimiento, ya que para trabajar a frecuencias más

altas normalmente se necesita un mayor voltaje. Por esto, al reducir la frecuencia también puede disminuir el voltaje, lo que afecta directamente la potencia dinámica debido a la relación cuadrática de V en la ecuación.

Sin embargo, esta reducción de potencia por sí sola no asegura un menor consumo de energía, porque también interviene el tiempo de ejecución. Si la aplicación tarda más en finalizar al trabajar a una frecuencia menor, parte del ahorro obtenido puede perderse por ese tiempo adicional. Además, durante toda la ejecución sigue existiendo consumo asociado a la potencia estática. Por eso, el efecto energético de cambiar la frecuencia también depende de cómo responda la carga de trabajo.

1.1.2. Modelo Roofline y regímenes de ejecución

El modelo Roofline relaciona el rendimiento alcanzable de una carga con la capacidad de cómputo y el ancho de banda de memoria de la plataforma. La intensidad operacional determina cuál de estos dos límites condiciona el rendimiento y se incorpora al modelo como:

$$P = \min(P_{\text{pico}}, I \cdot BW_{\text{pico}}) \quad (1.2)$$

donde P_{pico} representa el rendimiento aritmético pico, BW_{pico} el ancho de banda pico de memoria e I la intensidad operacional [35]. Cuando el rendimiento queda limitado por $I \cdot BW_{\text{pico}}$, la carga se encuentra en régimen *memory-bound*. Si el límite corresponde a P_{pico} , la carga se encuentra en régimen *compute-bound*. La Figura 1.1 muestra esta separación entre ambos regímenes.

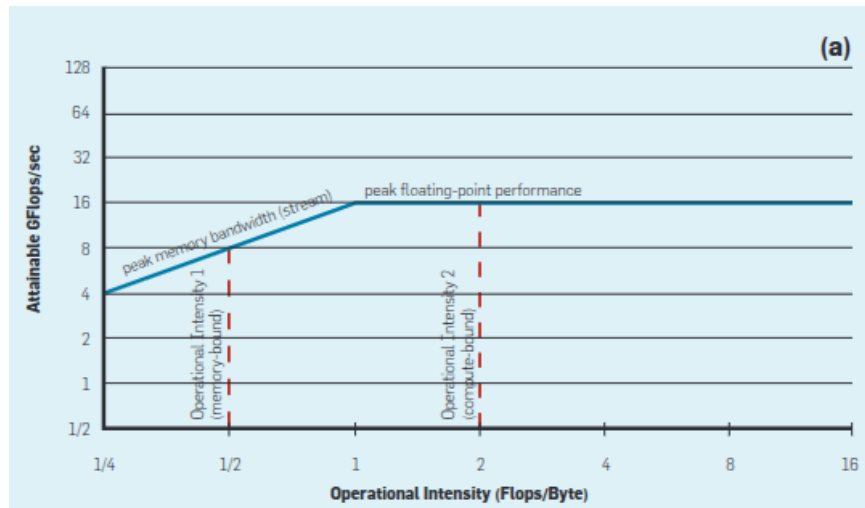


Figura 1.1: Modelo Roofline conceptual con el techo de ancho de banda de memoria y el techo de rendimiento de cómputo. Nota. Tomado de Williams et al. [35, Fig. 1(a)].

La intensidad operacional expresa la cantidad de operaciones realizadas por cada byte transferido desde el nivel de memoria considerado [35, p. 66]. Por ello, no depende únicamente del tamaño del conjunto de datos ni de si este cabe o no en caché. La reutilización de datos puede reducir el tráfico hacia memoria y aumentar la intensidad operacional, incluso cuando el conjunto de trabajo supera la capacidad de la caché [14, Secs. 3–4]. El modelo establece así un límite de rendimiento para cada intensidad, aunque una carga puede permanecer por debajo de ese límite por otros efectos de la ejecución.

La separación entre los dos regímenes ocurre en el punto de inflexión, también denominado *ridge point*. En ese punto, el techo de cómputo y el límite impuesto por el ancho de banda tienen el mismo valor, por lo que el umbral se obtiene como:

$$I_{\text{ridge}} = \frac{P_{\text{pico}}}{BW_{\text{pico}}} \quad (1.3)$$

Una intensidad inferior a I_{ridge} corresponde al régimen *memory-bound*, mientras que una intensidad superior corresponde al régimen *compute-bound* [35]. El *ridge point* funciona como frontera entre ambos regímenes y no como una frecuencia óptima de ejecución. Si alguno de los techos cambia, también puede cambiar la posición de esta frontera.

El techo de cómputo depende de la capacidad aritmética disponible y puede variar con la precisión de las operaciones y con la frecuencia de las unidades de cómputo [3]. El ancho de banda de memoria pertenece a otro dominio de la arquitectura y no tiene que cambiar en la misma proporción cuando se modifica la frecuencia [16, 2]. Por ello, el valor de I_{ridge} utilizado para clasificar una carga debe corresponder a la precisión y al punto de operación bajo el cual se realiza la observación. Los valores nominales sirven como referencia, pero la clasificación requiere techos representativos de las condiciones de ejecución consideradas [27].

La intensidad operacional observada se obtiene a partir del trabajo aritmético y del tráfico hacia memoria medidos durante el mismo intervalo. Para un intervalo i se calcula como:

$$I_i = \frac{\text{FLOPs}_i}{B_i^{\text{DRAM}}} \quad (1.4)$$

donde FLOPs_i representa las operaciones realizadas y B_i^{DRAM} los bytes transferidos hacia o desde memoria principal durante ese intervalo [35]. Los eventos utilizados para obtener los FLOPs dependen de la microarquitectura y deben interpretarse de acuerdo con la semántica de la PMU [34]. El tráfico de DRAM se obtiene desde contadores asociados al controlador

de memoria, cuyo ámbito puede ser diferente al de los contadores de núcleo. Ambas mediciones deben corresponder al mismo intervalo para que la intensidad operacional conserve su significado físico.

1.1.3. Telemetría microarquitectónica con PMU

La clasificación en CPU requiere señales que describan el comportamiento de la ejecución sin reconstruir en línea la intensidad operacional. Las Unidades de Monitorización de Rendimiento (PMU), proporcionan esa observabilidad mediante contadores de hardware que acumulan eventos como instrucciones retiradas, ciclos y actividad de la jerarquía de memoria. Estos contadores pueden ser fijos, asociados a eventos arquitectónicos predefinidos, o programables, configurados para registrar eventos específicos de la arquitectura [33, 18]. La siguiente figura resume esta organización:

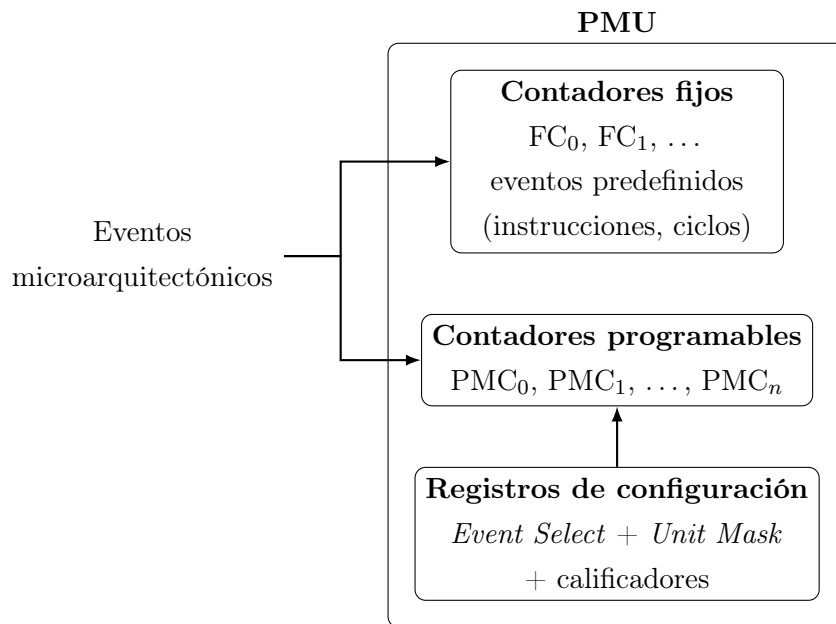


Figura 1.2: Organización conceptual de una Unidad de Monitorización de Rendimiento (PMU) con contadores fijos y programables. Fuente: elaboración propia.

Sin embargo, PMU no interpreta el comportamiento de la aplicación, sino que entrega conteos cuyo valor depende del evento y del dominio observado. Por ello, el mismo valor numérico solo adquiere significado después de considerar qué recurso cuenta, durante qué intervalo y con qué alcance.

Un problema que presentan estos contadores es la multiplexación, que ocurre cuando se solicitan más eventos de los que el hardware puede medir al mismo tiempo. En estas

condiciones, el sistema alterna distintos grupos de eventos y cada contador permanece activo solo durante una parte de la ventana de medición, por lo que el valor registrado se ajusta teniendo en cuenta el tiempo durante el cual estuvo realmente activo [25, 24]. Aunque este mecanismo permite trabajar con más eventos de los disponibles físicamente, la estimación puede perder precisión si la aplicación cambia de fase durante los periodos en los que un evento no está siendo medido, lo que hace conveniente limitar la cantidad de señales observadas de forma simultánea.

Otro detalle que también es importante tener en cuenta es el dominio de los contadores, puesto que existen contadores de núcleo y contadores de *uncore* que difieren en su nivel de acceso y alcance. Los primeros observan recursos asociados a un núcleo y pueden vincularse con la ejecución de una tarea, mientras que los de *uncore* cubren componentes compartidos del zócalo, entre ellos el controlador de memoria integrado y partes de la interconexión, desde donde se puede observar el movimiento efectivo de datos hacia memoria principal. Sin embargo, al tratarse de medidas con alcance compartido, el tráfico registrado por los contadores de *uncore* no puede atribuirse directamente a un proceso individual sin condiciones adecuadas de aislamiento. Esta diferencia entre una medida local de núcleo y una medida compartida también hace que ambas fuentes puedan trabajar con granularidades temporales distintas, por lo que no deben combinarse fila a fila sin establecer antes una correspondencia temporal. La siguiente figura muestra esta diferencia estructural.

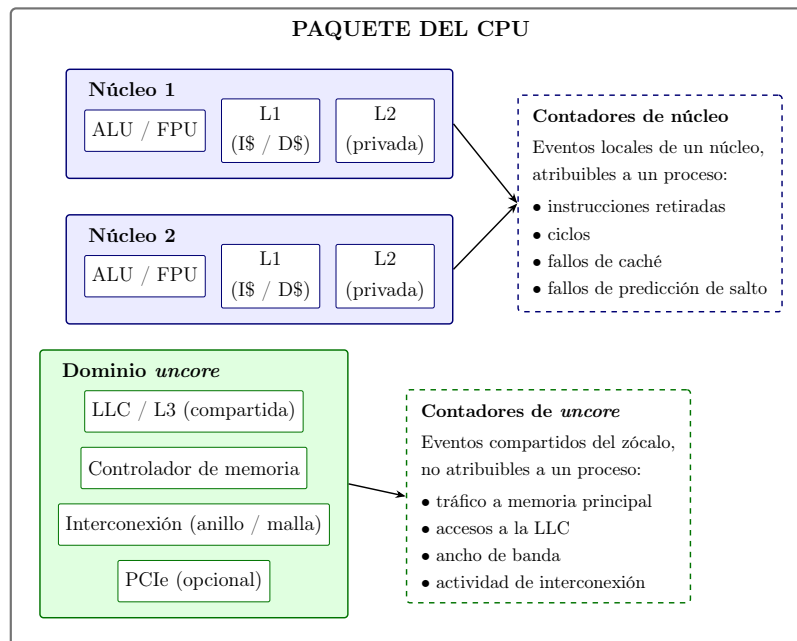


Figura 1.3: Ámbito de atribución de los contadores de núcleo y de *uncore* dentro del paquete del procesador. Fuente: Elaboración propia.

1.1.4. Espacios de ejecución *user-space* y *kernel-space*

Los sistemas operativos modernos emplean distintos niveles de privilegio para regular el acceso a los recursos del sistema, lo que permite distinguir entre los procesos que se ejecutan en *user-space* y aquellos que operan en *kernel-space*. Los procesos en *user-space* trabajan con acceso restringido, mientras que el *kernel-space* concentra las operaciones que requieren acceso directo a recursos protegidos, como la gestión de memoria, la entrada/salida o ciertas funciones del hardware [21, ch. 2]. Debido a esta separación, una aplicación en espacio de usuario no accede directamente a estos recursos y depende de las interfaces que el sistema operativo expone para realizar operaciones controladas [26], y es esta condición la cual también influye en la ubicación de programas de monitoreo y control, ya que determina la forma en que pueden consultar contadores, leer información de energía o solicitar cambios de frecuencia sin modificar estructuras internas del kernel. La siguiente figura detalla más la estructura de estos espacios y la relación conceptual entre ellos y el hardware físico.



Figura 1.4: Relación conceptual entre *user-space*, *kernel-space* y el hardware físico. Fuente: Elaboración propia.

1.1.5. Interfaces de observabilidad

Algunos sistemas operativos proporcionan interfaces que permiten a los programas en espacio de usuario consultar información del hardware sin acceder directamente a sus estructuras internas. En Linux, los eventos de la PMU pueden obtenerse mediante `perf_event_open`, una interfaz que permite configurar los contadores disponibles y consultar sus valores durante la ejecución [24], aunque su disponibilidad depende del soporte del procesador y de los permisos definidos en el sistema.

Estos conteos permiten observar la actividad del procesador, pero no cubren por sí solos la información energética. Para ello puede utilizarse Intel RAPL (*Running Average Power Limit*), un mecanismo incorporado en procesadores Intel que expone contadores de energía para distintos dominios, entre ellos el paquete del procesador y, cuando está disponible, la memoria DRAM [11, 30]. Sus valores representan energía acumulada durante la ejecución, por lo que pueden utilizarse para conocer el consumo asociado a un intervalo y posteriormente relacionarlo con métricas como el EDP.

1.1.6. DVFS, P-states y mecanismos de actuación

El escalado dinámico de voltaje y frecuencia (*Dynamic Voltage and Frequency Scaling*, DVFS) permite modificar el punto operativo del procesador para adaptar su capacidad de cómputo y su consumo energético a las necesidades de la carga. Estos cambios se apoyan en mecanismos de gestión de potencia que reciben una solicitud de rendimiento y la materializan sobre el hardware [17].

En CPU, los *Performance States* o P-states representan distintos niveles de rendimiento mientras el procesador permanece activo en C0. P0 corresponde al mayor nivel de rendimiento y los estados siguientes reducen progresivamente la capacidad disponible. Los C-states, en cambio, representan estados de inactividad con distintos compromisos entre ahorro energético y latencia de reactivación. Sobre los P-states operan los *governors*, políticas software que deciden qué nivel de rendimiento solicitar según el estado del sistema. En Linux, **performance** y **powersave** favorecen extremos distintos del rango disponible, mientras que **ondemand**, **conservative** y **schedutil** ajustan la solicitud de forma dinámica; **userspace**, por su parte, permite que una aplicación solicite explícitamente el nivel deseado [17, 23].

La Figura 1.5 reúne estos dos niveles de funcionamiento. En la parte (a) se representa la relación entre los estados de rendimiento y de inactividad del procesador, mientras que la parte (b) muestra el recorrido de una solicitud desde la interfaz de espacio de usuario, pasando por las políticas y los controladores del sistema, hasta los mecanismos que actúan sobre el hardware. Esta separación permite entender también por qué una decisión de rendimiento no se traduce necesariamente de forma inmediata en la frecuencia observada.

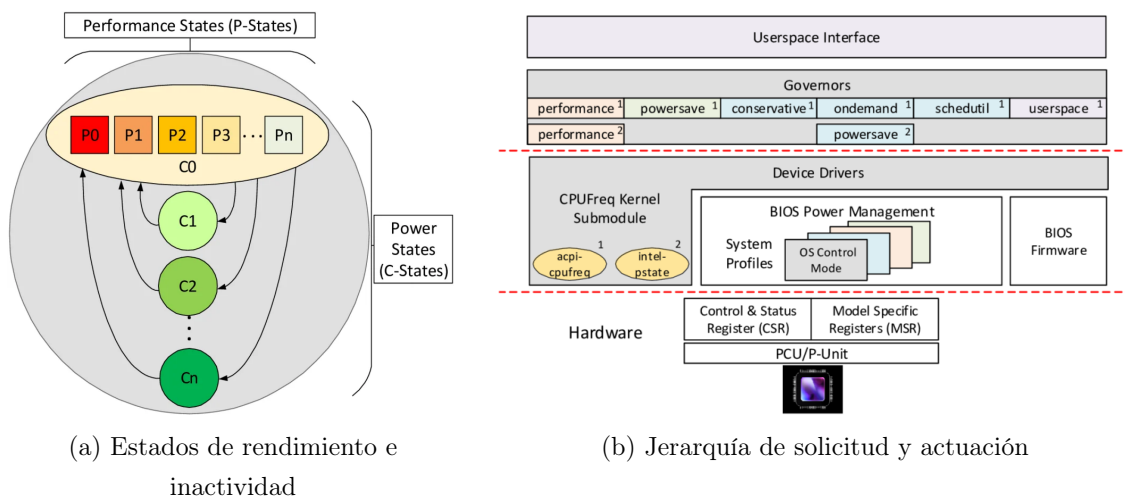


Figura 1.5: Relación entre los estados de rendimiento e inactividad del procesador y la jerarquía mediante la cual una política de rendimiento solicita cambios que posteriormente son materializados por el hardware. Fuente: Tomado de Hebbar y Milenković [17].

En GPU, DVFS mantiene el mismo principio general de ajustar el punto de operación del dispositivo, aunque la actuación suele realizarse sobre dominios de reloj distintos, principalmente el reloj de cómputo y el de memoria. Estos cambios dependen de los mecanismos de gestión propios de la arquitectura y del controlador, que determinan los niveles de frecuencia disponibles y la forma en que una solicitud se aplica sobre el hardware [16, 2]. Al igual que en CPU, la transición entre niveles requiere un tiempo de actuación, por lo que el cambio de frecuencia no es instantáneo [31].

1.1.7. Cargas de calibración y modelos de clasificación ligera

Las aplicaciones de cómputo pueden combinar fases con demandas distintas sobre las unidades de procesamiento y la jerarquía de memoria, por lo que su comportamiento puede observarse mediante cargas o unidades de ejecución más acotadas. Entre ellas se encuentran los *kernels*, que corresponden a operaciones computacionales bien delimitadas, como una multiplicación de matrices, una transformada de Fourier o un stencil, y permiten observar con mayor claridad cómo una operación utiliza los recursos del sistema [35]. También se encuentran los *benchmarks*, que proporcionan cargas utilizadas para medir o comparar el comportamiento de una plataforma bajo condiciones reproducibles, y los *microbenchmarks*, que se construyen para aislar propiedades más específicas, como el ancho de banda de memoria, la latencia de acceso o la capacidad de ejecución aritmética [15].

Las observaciones obtenidas durante estas ejecuciones pueden representarse mediante un conjunto de características y una clase asociada, relación que puede aprenderse mediante clasificación supervisada para después inferir la clase de nuevas observaciones. Cuando este proceso debe realizarse con un costo reducido, conviene trabajar con modelos de clasificación ligera, entendidos como modelos cuya complejidad permite mantener un tiempo de inferencia bajo sobre un número reducido de características.

Existen varios modelos ligeros y no todos representan de la misma forma la relación entre las características. Por ejemplo, la regresión logística establece una frontera lineal y de bajo costo, por lo que sirve como referencia cuando la separación entre clases puede describirse mediante relaciones relativamente simples [19, Ch. 4]. Un árbol de decisión, en cambio, permite introducir relaciones no lineales mediante divisiones sucesivas de las características y conserva una estructura fácil de interpretar, aunque su complejidad aumenta a medida que crece la profundidad del árbol [19, Ch. 8].

A partir de los árboles también pueden construirse modelos de ensamble, como Random Forest, que combina varios árboles generados mediante muestreo y selección aleatoria de

características, reduciendo la dependencia de una única partición y permitiendo representar interacciones no lineales entre las variables [6]. Extra-Trees parte de una estructura similar, pero introduce mayor aleatoriedad en la selección de los puntos de corte [12], por lo que ambos modelos pueden trabajar con relaciones más complejas que las representadas por una frontera lineal.

XGBoost también utiliza árboles de decisión, pero los construye de manera secuencial, de modo que cada nuevo árbol busca corregir parte de los errores acumulados por los anteriores [9]. Esto le permite representar relaciones más complejas, aunque incorpora un mayor número de parámetros y su costo depende de la configuración del ensamble. En conjunto, estas familias cubren distintos niveles de complejidad, desde una frontera lineal hasta conjuntos de árboles capaces de representar interacciones no lineales, manteniendo como consideración común el costo necesario para realizar la inferencia.

1.1.8. Hiperparámetros, Optuna y validación

Los hiperparámetros controlan aspectos de la estructura y del proceso de entrenamiento de un modelo, por lo que su elección puede modificar tanto la capacidad de ajuste como su complejidad. En modelos como Random Forest, Extra-Trees o XGBoost, estos valores pueden incluir cantidades de árboles, profundidades, tasas de aprendizaje u otros parámetros de configuración, lo que genera espacios de búsqueda con combinaciones muy diferentes entre sí. Explorar todas las posibilidades mediante una rejilla puede resultar costoso, de modo que suelen emplearse métodos de optimización que orientan la búsqueda hacia configuraciones más prometedoras.

Optuna es un marco de optimización de hiperparámetros que realiza esta exploración de forma secuencial [1]. Entre sus estrategias se encuentra el estimador de Parzen estructurado (TPE), que utiliza los resultados obtenidos en evaluaciones anteriores para proponer nuevas configuraciones con mayor probabilidad de mejorar el objetivo [5]. De esta manera, la búsqueda no necesita recorrer todas las combinaciones posibles y puede adaptarse a espacios que mezclan variables enteras, continuas y categóricas.

La optimización de hiperparámetros debe mantenerse separada de la evaluación final del modelo, ya que utilizar la misma información para seleccionar y evaluar una configuración puede producir estimaciones demasiado favorables. La validación anidada permite mantener esta separación mediante un proceso interno para seleccionar la configuración y otro externo para estimar su capacidad de generalización [8, 28]. Cuando las observaciones pertenecen a una misma familia de carga o comparten un origen común, conservar esa relación durante la

partición evita que ejemplos muy similares aparezcan simultáneamente en entrenamiento y prueba.

También debe evitarse la fuga de información desde las características de entrada. Si una variable utilizada por el modelo contiene directa o indirectamente la información con la que se construyó la etiqueta, el clasificador puede aprender a reconstruir esa regla en lugar de identificar el patrón a partir de señales independientes [20]. Por ello, la información utilizada como referencia para definir la clase debe mantenerse separada de las características disponibles para la inferencia.

1.1.9. Producto Energía–Retardo (EDP)

Los cambios de frecuencia mediante DVFS afectan tanto el consumo de energía como el tiempo de ejecución, por lo que reducir una de estas magnitudes no significa necesariamente que el resultado sea mejor. Es por esto que el Producto Energía–Retardo (*Energy–Delay Product*, EDP) relaciona ambas en una sola expresión:

$$EDP = E \cdot T$$

donde E representa la energía consumida y T el tiempo de ejecución [22]. Según el valor obtenido, un EDP menor representa una relación más favorable entre energía y tiempo. Sin embargo, ahorrar energía no siempre resulta conveniente si para lograrlo la ejecución tarda mucho más, ya que durante ese tiempo adicional el sistema continúa consumiendo energía, incluida la asociada a la potencia estática.

El EDP reúne la energía y el tiempo en un solo valor, aunque ambas magnitudes todavía deben revisarse por separado, porque dos configuraciones pueden obtener un EDP parecido aun cuando una consuma menos energía y la otra tenga un menor tiempo de ejecución. Por eso, la métrica permite comparar distintos puntos de operación teniendo en cuenta ambos efectos al mismo tiempo.

1.2. Estado del Arte

La gestión energética en sistemas de alto rendimiento ha evolucionado desde enfoques estáticos de reducción de potencia hacia estrategias cada vez más adaptativas, orientadas a preservar el rendimiento sin incurrir en costos energéticos desproporcionados. Dentro de este proceso, el escalado dinámico de voltaje y frecuencia (DVFS) se ha consolidado como

uno de los mecanismos más estudiados para modificar el punto operativo de procesadores y aceleradores. Su aplicación, sin embargo, no conduce necesariamente al mismo resultado para todas las cargas de trabajo, ya que una reducción de frecuencia puede disminuir la potencia instantánea, pero también prolongar el tiempo de ejecución y anular el beneficio energético esperado. Esta dependencia ha llevado a que buena parte de la investigación pase de aplicar DVFS de forma uniforme a buscar mecanismos capaces de reconocer el comportamiento de la carga y ajustar la frecuencia de acuerdo con sus características.

Las primeras aproximaciones que analizaron DVFS sobre regiones concretas de una aplicación ya mostraban que mantener una misma frecuencia durante toda la ejecución podía desaprovechar oportunidades de ahorro. Calore et al. [7], al estudiar distintos kernels y una aplicación HPC completa sobre una CPU Intel Haswell y una GPU NVIDIA K80, observaron que el efecto energético variaba entre regiones y arquitecturas. En lugar de mantener un único estado operativo, evaluaron cambios de frecuencia a nivel de función, mostrando que la conveniencia de cada configuración dependía del comportamiento de la región ejecutada y de la plataforma sobre la cual se realizaba el cálculo.

Esta dependencia de la carga llevó a buscar formas de anticipar su respuesta sin recorrer todas las frecuencias disponibles. Guerreiro et al. [16] estudiaron aplicaciones GPGPU a partir de métricas recolectadas en una configuración inicial y utilizaron esa caracterización para estimar cómo responderían ante cambios en los dominios de cómputo y memoria. Con ello redujeron la necesidad de explorar exhaustivamente el espacio DVFS y mostraron que la información de rendimiento podía utilizarse para agrupar aplicaciones según su sensibilidad a la frecuencia y orientar la selección hacia configuraciones energéticamente favorables.

Sobre una idea similar, Ali et al. [2] propusieron un método automatizado y portable para seleccionar frecuencias de GPU a partir de métricas obtenidas inicialmente en la configuración predeterminada del dispositivo. Con esta caracterización construyeron modelos de potencia y tiempo de ejecución para estimar el comportamiento de la aplicación en otros estados DVFS sin ejecutarla en cada uno de ellos. La selección consideró de forma conjunta el consumo energético y la degradación del desempeño, evitando interpretar una reducción de potencia como una mejora cuando venía acompañada de un incremento importante en el tiempo de ejecución. En las plataformas evaluadas, la propuesta reportó reducciones promedio de energía de 29.6 % con una pérdida de rendimiento de 5.2 % sobre NVIDIA GA100 y de 22.6 % con una pérdida de 4.7 % sobre NVIDIA GV100 [2].

Aunque estos enfoques permiten reducir el espacio de búsqueda, parten principalmente de una caracterización con la que se anticipa el comportamiento posterior de la aplicación.

Cuando la carga cambia durante la ejecución aparece otra necesidad, reconocer el régimen que domina en cada momento. Antici et al. [4], mediante MCBound, abordaron esta identificación como un problema de clasificación entre cargas *memory-bound* y *compute-bound*, utilizando información de ejecuciones HPC para caracterizar automáticamente nuevos trabajos. Su evaluación sobre datos del supercomputador Fugaku mostró que esta separación puede realizarse con una puntuación F1-macro de al menos 0.89 y con un costo reducido para el funcionamiento del sistema [4]. Este resultado aporta evidencia de que la naturaleza computacional de una carga puede inferirse automáticamente sin depender únicamente de una clasificación manual basada en el conocimiento previo de la aplicación.

La clasificación de MCBound resulta especialmente cercana al problema de identificación del régimen, aunque su objetivo no es gobernar directamente los estados DVFS de CPU o GPU. Aun así, permite separar dos partes que aparecen de manera recurrente en los trabajos revisados, primero reconocer el comportamiento de la carga y luego decidir cómo utilizar esa información para modificar el punto de operación. La segunda parte ha sido explorada desde distintas aproximaciones orientadas a adaptar la frecuencia durante la propia ejecución.

Simsek et al. [29], por ejemplo, estudiaron esta actuación dinámica en simulaciones astrofísicas ejecutadas sobre GPU. Los autores instrumentaron SPH-EXA para medir el comportamiento energético de diferentes regiones y modificar la frecuencia del acelerador en puntos determinados de la ejecución. Frente a una política de frecuencia uniforme, esta adaptación permitió reducir el consumo energético por GPU hasta en 7.82 %, con una degradación del rendimiento de aproximadamente 2.95 %. El trabajo muestra que existe un beneficio potencial al cambiar la frecuencia durante la ejecución, aunque la decisión permanece asociada a regiones previamente conocidas e instrumentadas dentro de la aplicación.

Un paso más hacia un control *online* completo aparece en DRLCAP, un marco de *frequency capping* para GPU basado en aprendizaje por refuerzo profundo. DRLCAP monitoriza información a nivel de sistema, detecta cambios de fase y ajusta el límite de frecuencia sin intervención manual, logrando una reducción promedio de 22 % en la energía de GPU con menos de 3 % de degradación de rendimiento en arquitecturas NVIDIA [32]. Este resultado muestra que el control de frecuencia durante la ejecución puede automatizarse a partir de señales observadas en línea, aunque mediante un enfoque de aprendizaje por refuerzo profundo y concentrado exclusivamente en GPU.

Los trabajos revisados muestran una transición desde la aplicación de configuraciones DVFS sobre regiones conocidas hacia mecanismos que utilizan información de ejecución para anticipar o adaptar la frecuencia. En este recorrido, la caracterización de la carga ha permi-

tido reducir búsquedas exhaustivas, mientras que enfoques como MCBound muestran que el régimen computacional también puede inferirse mediante clasificación. De forma paralela, los trabajos de Simsek et al. [29] y Wang et al. [32] evidencian que la frecuencia puede ajustarse durante la propia ejecución, aunque mediante estrategias ligadas a regiones previamente instrumentadas o a modelos de aprendizaje más complejos.

Aun con estos avances, gran parte de los enfoques recientes de predicción y control dinámico se concentra en GPU, mientras que CPU y GPU presentan diferencias tanto en sus mecanismos de frecuencia como en las señales disponibles para observar la ejecución. Calore et al. [7] consideran ambas arquitecturas, pero Guerreiro et al. [16], Ali et al. [2], Simsek et al. [29] y Wang et al. [32] profundizan principalmente en aceleradores gráficos. Esta diferencia deja espacio para estudiar estrategias que integren ambas plataformas y utilicen una caracterización de bajo costo como apoyo para decisiones DVFS durante la ejecución, sin depender de una instrumentación específica de cada aplicación.

Bibliografía

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.
- [2] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen. An automated and portable method for selecting an optimal gpu frequency. *Future Generation Computer Systems*, 149:71–88, 2023. doi: 10.1016/j.future.2023.07.011.
- [3] Oscar Antepara, Leonid Oliker, and Samuel Williams. Roofline analysis of tightly-coupled CPU-GPU superchips: A study on MI300A and GH200. In *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1205–1216, 2025. doi: 10.1145/3731599.3767497.
- [4] F. Antici, A. Bartolini, Z. Kiziltan, O. Babaoglu, and Y. Kodama. Mcbound: An online framework to characterize and classify memory/compute-bound hpc jobs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, Atlanta, GA, USA, 2024. doi: 10.1109/SC41406.2024.00062.
- [5] James S. Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems*, volume 24, pages 2546–2554, 2011.
- [6] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001. doi: 10.1023/A:1010933404324.
- [7] E. Calore, A. Gabbana, S. F. Schifano, and R. Tripiccione. Evaluation of dvfs techniques on modern hpc processors and accelerators for energy-aware applications. *Concurrency and Computation: Practice and Experience*, 29(12):e4143, 2017. doi: 10.1002/cpe.4143.

- [8] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.
- [9] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
- [10] M. T. Costa et al. Characterizing the impact of gpu power management on an exascale system. In *Proc. SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*, pages 1524–1533, New York, NY, USA, 2025. doi: 10.1145/3731599.3767702.
- [11] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le. RAPL: Memory power estimation and capping. In *Proc. 16th ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED)*, pages 189–194, 2010. doi: 10.1145/1840845.1840883.
- [12] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006. doi: 10.1007/s10994-006-6226-1.
- [13] R. Gonzalez, B. M. Gordon, and M. A. Horowitz. Supply and threshold voltage scaling for low power CMOS. *IEEE Journal of Solid-State Circuits*, 32(8):1210–1216, 1997. doi: 10.1109/4.604018.
- [14] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):1–25, 2008. doi: 10.1145/1356052.1356053. Article 12.
- [15] B. Gregg. *Systems Performance: Enterprise and the Cloud*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2020.
- [16] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás. DVFS-aware application classification to improve GPGPUs energy efficiency. *Parallel Computing*, 83:93–117, 2019. doi: 10.1016/j.parco.2018.02.001.
- [17] R. Hebbar and A. Milenković. PMU-events-driven DVFS techniques for improving energy efficiency of modern processors. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, 7(1):1–31, 2022. doi: 10.1145/3538645.

- [18] Intel Corporation. Intel 64 and ia-32 architectures software developer’s manual, volume 3b: System programming guide, part 2 (performance monitoring, nmi watchdog interaction with fixed-function and general-purpose counters), 2026. URL <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>. Accessed: Aug. 12, 2026.
- [19] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer Texts in Statistics. Springer, 2nd edition, 2021. doi: 10.1007/978-1-0716-1418-1.
- [20] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, 2012. doi: 10.1145/2382577.2382579.
- [21] M. Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, San Francisco, CA, USA, 2010.
- [22] J. H. Laros III, K. Pedretti, S. M. Kelly, W. Shu, K. Ferreira, J. Van Dyke, and C. Vaughan. Energy delay product. In *Energy-Efficient High Performance Computing: Measurement and Tuning*, SpringerBriefs in Computer Science, chapter 6, pages 51–55. Springer, London, UK, 2013. doi: 10.1007/978-1-4471-4492-2_9.
- [23] Linux Kernel Documentation. CPU Performance Scaling, 2026. URL <https://docs.kernel.org/admin-guide/pm/cpufreq.html>. Accessed: Sep. 5, 2026.
- [24] Linux man-pages project. perf_event_open(2) — linux manual page, 2026. URL https://man7.org/linux/man-pages/man2/perf_event_open.2.html. Accessed: Aug. 29, 2026.
- [25] T.-Y. Liu, J. Guo, and B. Huang. Efficient cross-platform multiplexing of hardware performance counters via adaptive grouping. *ACM Transactions on Architecture and Code Optimization*, 21(1):1–26, 2024. doi: 10.1145/3629525.
- [26] S. McCarty. Architecting containers part 1: Why understanding user space vs. kernel space matters. Red Hat Blog, 2015. URL <https://www.redhat.com/en/blog/architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters>. Accessed: Apr. 3, 2025.
- [27] National Energy Research Scientific Computing Center (NERSC). Roofline performance model. NERSC Documentation, 2026. URL <https://docs.nersc.gov/tools/performance/roofline/>. Accessed: Aug. 7, 2026.

- [28] David R. Roberts, Volker Bahn, Simone Ciuti, Mark S. Boyce, Jane Elith, Gurutzeta Guillera-Arroita, Severin Hauenstein, José J. Lahoz-Monfort, Boris Schröder, Wilfried Thuiller, David I. Warton, Brendan A. Wintle, Florian Hartig, and Carsten F. Dormann. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8):913–929, 2017. doi: 10.1111/ecog.02881.
- [29] O. S. Simsek, J.-G. Piccinali, and F. M. Ciorba. Increasing energy efficiency of astrophysics simulations through gpu frequency scaling. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1826–1834, Atlanta, GA, USA, 2024. doi: 10.1109/SCW63240.2024.00229.
- [30] P. Thamm and U. Leser. Strategies to measure energy consumption using RAPL during workflow execution on commodity clusters. *arXiv preprint arXiv:2505.09375*, 2025.
- [31] D. Velicka, O. Vysocky, and L. Riha. Methodology for GPU frequency switching latency measurement. In *Proc. 2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 830–839, 2025. doi: 10.1109/IPDPSW66978.2025.00133.
- [32] Y. Wang, M. Hao, H. He, W. Zhang, Q. Tang, X. Sun, and Z. Wang. DRLCap: Runtime GPU frequency capping with deep reinforcement learning. *IEEE Transactions on Sustainable Computing*, 9(5):712–726, 2024. doi: 10.1109/TSUSC.2024.3361596.
- [33] V. M. Weaver. Self-monitoring overhead of the linux perf event performance counter interface. In *Proc. IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 102–111, 2015.
- [34] V. M. Weaver and S. A. McKee. Can hardware performance counters be trusted? In *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, pages 141–150, Seattle, WA, USA, 2008. doi: 10.1109/IISWC.2008.4636099.
- [35] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Repositorio y recursos complementarios

Repositorio público del proyecto:

<https://github.com/AdrianCCRS/hyperion>